

Mikhail LeTal: a chess agent for AI Chessathon 2026

Design, evidence and rating estimate

Alexander Urvancev

September 2026 (draft for build v0.1.0; updated with each build)

Abstract

Mikhail LeTal is a chess agent written for AI Chessathon 2026 by a one-person team. The shipped build (v0.1.0, “Stage 0”) is plain Python 3.12 on top of the `python-chess` library, which supplies the board and the rules of chess; the search, the evaluation, the time management, the game-state tracking and the safety wrapper are our own and are described here so that every line can be explained. Locally the build won all 300 fuzz games against the starter kit’s random mover and all 200 against its one-ply greedy agent, with no crash, illegal move, flag fall or start-up failure (source: `RESULTS.md`, rows `fuzz-random-v0.1` and `fuzz-greedy-v0.1`), and scored 98.8% ($\pm 2.4\%$; $+39 = 1 - 0$) in 40 games at the real clock against the starter’s two-ply minimax baseline, again with no failure termination (source: `RESULTS.md`, row `v0.1-vs-minimax-real-clock`). Three measurements are still open and are marked in red throughout: the per-move log inspection is done (Section ??: solo pass, slowest move 8 429 ms, minimum clock 42 958 ms, one 2 ms overshoot); **[PENDING: first platform validation log and the time calibration derived from it]**; **[PENDING: Stage 3 rating estimate on the CCRL, FIDE and chess.com scales]**. Every other number in this document is a local measurement quoted from `docs/RESULTS.md` or a constant quoted from the code and its provenance files; nothing is estimated from memory.

1 The competition and what was built

Format. AI Chessathon 2026 is a competition for chess agents written by student teams. A team uploads a zip whose root `agent.py` exposes one function, `get_move(fen, time_left_ms)`, returning a move in UCI notation (for example `e2e4`, or `e7e8q` for a promotion). The platform plays the latest build that passed validation on a rated ladder (hourly rounds, 08:00–22:00 London). The ladder only seeds a 13-round Swiss tournament over locked builds on the afternoon of Thursday 11 September; the Swiss decides qualification for the London final on Saturday 12 September, where each finalist team walks a judge through how its agent was built. Uploads close at 11:00 London on 11 September.

Clock and machine. Each side has 120 s plus 0.5 s per move, measured in wall time by the runner from the moment it writes the request to the moment it reads the reply, so process wake-up, FEN parsing and board construction are on our clock. The agent runs as one process per game, suspended while the opponent thinks, on one core of an AMD EPYC 9V74 at 2.60 GHz with 2 GB of RAM, no network and no GPU; it gets 90 s at start-up before the clock runs. The Python environment is fixed: Python 3.12 with `python-chess 1.11.2`, `numpy 2.5.2`, `numba 0.67.0`, `torch 2.13.0` (CPU) and `onnxruntime 1.29.0`; nothing else can be installed and native binaries are rejected. A game is lost outright on an illegal move, a crash, running out of memory, missing the 90 s start-up budget, or a flag fall (unless the opponent cannot mate, which is a draw). Draws follow FIDE rules and the referee claims threefold repetition and fifty-move draws automatically; a game that reaches 600 plies (half-moves, counting the opening position) is a

draw. Every game starts from a curated near-level opening position, not from the initial position, and the set of openings is not published (source: brief, §3; AGENTS.md).

What was built. The build described here is Stage 0 of a staged plan (Section ??). It ships `agent.py` (the wrapper), the package `mikhail_letal` (evaluation, search, timing, game state, fallback), `weights/pst.json` (the evaluation tables) and `weights/PROVENANCE.json`: 23 333 bytes zipped, 66 825 bytes unzipped (source: RESULTS.md, row “zip”). It reaches depth 5–6 with selective depth 18–32 at the real clock on the development machine, at 50–61 thousand nodes per second (source: RESULTS.md, rows “depth at 120 s clock” and “node rate, middlegame”).

Our result. Local: 300/300 against `baselines/random` and 200/200 against `baselines/greedy`, all by checkmate, zero failure terminations (Section ??). Against `baselines/minimax` at the real clock (120 s + 0.5 s, four games at a time): 40 games, +39 =1 –0, 98.8 % (± 2.4 %), 39 by checkmate and one draw by threefold repetition, no flag (source: RESULTS.md, row `v0.1-vs-minimax-real-clock`); the brief’s Stage 0 bar was a score of at least 90 % with zero flags. The per-move log of that run (slowest move, minimum clock reached) is the solo pass in Section ?? shows the budget holding (worst overshoot 2 ms). On the platform: **[PENDING: first validation log (init time, per-move times, clock left)]** and **[PENDING: ladder rating of build v0.1.0]**.

2 Rules-compliance statement

- No third-party engine, wrapper, port or translation is shipped or consulted at runtime. The only library used at runtime is `python-chess 1.11.2`, which is part of the platform’s fixed stack and provides board representation, move generation and the rules, not chess knowledge.
- No published network is shipped; Stage 0 ships no network at all. No positions labelled by any engine were used: Stage 0 has no tuned parameters (Section ??).
- No lookup database of another engine’s moves or evaluations ships in the zip. There is no opening book and no tablebase in this build (Section ??).
- Memorised-code risk. An engine written with a language model can reproduce code or tables it has seen, and Sunfish, whose tables are famous, is a house bot on this ladder. Three measures were taken. (1) Every evaluation table is generated by `tools/gen_pst.py` from a parametric geometric prior (Section ??): each entry is a formula of the square’s geometry times a named parameter, so the numbers are ours by construction and re-running the script reproduces them bit for bit. (2) Search and evaluation were written against a module contract (`docs/DESIGN.md`) from textbook descriptions of the algorithms, and a review pass looked for surface overlaps with known engines: the one found was the class-and-method pair `Searcher.search`, which Sunfish also uses; our class was renamed `Engine` even though everything behind it differs (source: DECISIONS.md, 2026-09-07 item F). (3) The generated tables were compared mechanically against the published tables of **Sunfish**, the Chess Programming Wiki’s “**simplified evaluation function**”, **Rustic Alpha 3**, **TSCP** and **VICE**: whole-table comparisons and affine comparisons (does any table of ours equal a published one after multiplying by a constant and adding a constant?) were all clean, and a row-by-row comparison found exactly **one coincidental row**: the endgame pawn table’s seventh rank is 50 in both ours and the CPW table, because our formula gives $14 \times 5^2 / 7 = 50$ there (source: DECISIONS.md, 2026-09-07 item I).
- Local measuring instruments. A rating-limited Stockfish (path read from the environment variable `YARDSTICK_ENGINE`) and the real Sunfish are used only as local sparring partners

for the rating estimate (Section ??); they live outside the repository, never enter the zip, never influence a move, and their source was not read to inform the design.

- Provenance. Every constant that ships has a row in Appendix ??, copied from `docs/PROVENANCE.md` and `weights/PROVENANCE.json`, which itself ships in the zip.
- Everything shipped is plain, type-annotated Python that passes `ruff check` and `mypy -strict` (source: `INTEGRATION_NOTES.md`, “Checks run”). No generated table ships without its generating script; nothing is minified or obfuscated.

3 Architecture and why

The plan has stages, and each stage replaces the previous shipped build only after beating it under the real clock with a 95 % interval above zero. Three architectures were on the table.

1. **python-chess board + our own search and evaluation in pure Python** (Stage 0, this build). `python-chess` is a proven implementation of the rules, so correctness risk is confined to our own code: search, evaluation, time management. This is what the first priority of the brief (never lose a game for free) demands from the first session.
2. **Own board representation, move generation, evaluation and search compiled with numba** (Stage 1, planned), with `python-chess` kept in the wrapper as the legality oracle and fallback. `numba` compiles at every game start, so its compile time must be measured from the zip and kept under 40 s locally against the 90 s start-up budget.
3. **A small self-trained NNUE-style network** evaluated inside the jitted search (Stage 4, expected to be skipped): `onnxruntime` cannot be called once per node from a jitted search, training data would have to be produced first, and the operator would rather explain a tuned hand-written evaluation than a rushed network.

A fourth option, jitting only the evaluation under a `python-chess` board, was rejected by arithmetic on the measured numbers: at 50–61 thousand nodes per second one node costs roughly 16–20 μ s, of which `evaluate()` is 3.4–3.8 μ s in the middlegame and 1.6 μ s in the endgame (source: `RESULTS.md`, row “`evaluate()` cost”), so the evaluation is at most a fifth of the node cost and `numba` cannot read `python-chess` objects without a conversion that would cost more than it saves. The speed has to come from replacing the board and the move generator, which is Stage 1.

architecture	node rate	depth at 120 s + 0.5 s	import time	peak RSS	status
Stage 0: python-chess + pure Python	50–61k nps (middlegame)	5–6 (seldepth 18–32)	0.033 s	42.4 MB after a 10 s search; 120 MB at the TT cap	shipped (v0.1.0)
Stage 1: numba engine	not built	not built	must be $\leq$ 40 s from the zip	must be < 500 MB	planned
Stage 4: NNUE-style net	not built	—	—	—	expected to skip

Table 1: Architectures and the measured numbers that decided the order. Stage 0 rows: `RESULTS.md`, rows “node rate, middlegame”, “depth at 120 s clock”, “import time”, “peak RSS after a 10 s search”, “RSS at the TT cap”. Stage 1 limits are the brief’s requirements, not measurements. “nps” is nodes per second; “seldepth” is the deepest ply reached including quiescence; RSS is resident memory.

Import time was measured three times by timing `import agent` with `time.perf_counter()` in a fresh interpreter and was 0.033 s each time, including a depth-2 warm-up search that runs at import so every code path has executed before the first real move (source: `RESULTS.md`, row

“import time”). After the 2026-09-07 review the node rate on a 3 s search from the initial position rose from 51.7k to 67.2k nps (+30 %, same move `g1f3`, depth 6/17) (source: RESULTS.md, row “review nps”); Section ?? lists what changed.

4 Board representation and move generation

Stage 0 uses python-chess’s `Board`: a bitboard representation (one 64-bit integer per piece type and per colour) with fully legal move generation, written and tested by the library’s authors and pinned by the platform at version 1.11.2. Quiescence uses `Board.generate_legal_captures()` so that only captures are generated at the leaves. Squares are indexed as python-chess does (`a1 = 0`, `h1 = 7`, `a8 = 56`); evaluation tables are stored from White’s point of view and a Black piece on square s reads the table at $s \oplus 56$ (a vertical mirror). Positions are identified by python-chess’s own repetition key, `Board._transposition_key()`, a tuple of the piece bitboards, the side to move, the castling rights and the en passant square; it is exactly what `Board.is_repetition` uses, so what we count as a repetition is what the referee counts (source: DECISIONS.md, 2026-09-06, “Position keys”). Two costs are accepted knowingly: the key is a private name (an upgrade of the library could remove it, but the platform pins 1.11.2), and it is a tuple rather than a Zobrist hash, which Stage 1 replaces.

Perft. A perft test counts the leaf nodes of the full move tree to a fixed depth and compares the count with a reference; it is the standard proof that a move generator is correct. Stage 0 has no move generator of its own, so there is nothing to perft against python-chess: legality agreement with the referee, which also uses python-chess, holds by construction. The evidence that the surrounding code uses the library correctly is the property-test set (each of the following returns a legal move within budget: all four promotion pieces, capture-promotions, under-promotion to a knight when it is the only mate, en passant for both colours, castling both ways for both colours, a stalemate trap, insufficient material, a single legal move, mate in one, halfmove clock 99, a would-be third repetition) and the 500 fuzz games with zero illegal moves (Section ??) (source: tests/test_properties.py; RESULTS.md fuzz rows). The Stage 1 move generator will be perft-tested against python-chess to depth 4 on the standard public positions and to depth 3 on every collected opening and on at least 2000 random playout positions, plus a legal-move-set equality test on at least 100 000 random positions, before it replaces the library (source: brief, §5 Stage 1 DoD); that table will be added here.

5 Search

The algorithm in one paragraph. The search is a *negamax* tree search: every score is measured from the side to move’s point of view, so the value of a position is the maximum over moves of the negated value of the resulting position. *Alpha-beta pruning* carries a window (α, β) of scores that can still matter to the ancestors and abandons a child whose value falls outside it. *Iterative deepening* runs the search to depth 1, then 2, then 3, and so on until the clock says stop; each iteration seeds the next with good first-try moves, and a fully searched answer is always available. At the leaves a *quiescence* search continues with captures (and queen promotions) only, because a static evaluation in the middle of an exchange is meaningless; the side to move may always *stand pat*, that is, accept the static evaluation instead of capturing. A *transposition table* (TT) remembers, per position key, the depth, score, score type (exact, lower bound or upper bound) and best move of an earlier visit, so that positions reached by different move orders are not searched twice. Scores are integers in centipawns (hundredths of a pawn); a mate delivered p plies from the root scores $100\,000 - p$, so shorter mates score higher. All of

this is in `mikhail_letal/search.py`.

Why these techniques and not others here. The platform gives one slow core and pure Python gives tens of thousands of nodes per second, so the tree is small (about 10^5 – 10^6 nodes per move at the real clock) and every technique that makes a small tree go deeper, or that keeps the clock check honest, matters more than it would at millions of nodes per second. Null-move pruning, late-move reductions, futility pruning, aspiration windows and quiescence delta pruning are deliberately absent from Stage 0: each will be added as a separate experiment with its own arena result so that the report can say what each is worth here (source: DECISIONS.md, 2026-09-06, “No null-move pruning”).

Table 2: Search techniques in build v0.1.0 and the local evidence for each. No per-feature arena ablation (feature on/off, games, interval) has been run yet; the effects listed are node-rate, latency and correctness measurements from the 2026-09-07 review. Elo ablations are Stage 1 work.

technique	what it does and why	measured effect (local, dev box)
Iterative deepening with soft and hard deadlines	No new iteration after the soft deadline; abort mid-iteration at the hard deadline and return the last completed iteration’s move (or the current one’s if a later root move already beat the first).	Depth 5–6 at 120 s + 0.5 s, 4–6 at 20 s + 0.2 s (source: RESULTS.md, “depth” rows).
Check extension	In check, search one ply deeper (capped at <code>MAX_PLY = 128</code>), applied before the TT probe so probe and store see the same depth (review item L: the probe had accepted entries one ply too shallow).	Correctness fix; pinned by a unit test in <code>tests/test_search.py</code> (check extension before the table probe).
Quiescence, stand pat first	Test the static evaluation against β before generating any capture; a position with no legal move is still scored as mate or stalemate, never evaluated. Before the change, captures were generated and then discarded at seven quiescence nodes in ten.	+43 % nps in isolation (reviewer); all review fixes together 51.7k $\rightarrow$ 67.2k nps, identical results (source: DECISIONS.md item K; RESULTS.md, “review nps”).
Evasion cap, <code>QS_EVASION_PLIES = 4</code>	In check inside quiescence, every evasion is searched for the first four quiescence plies; deeper checks stand pat like any other node so dense positions cannot explode.	Eight queens a side: one depth-1 iteration fell from hundreds of thousands of nodes to 33k (source: DECISIONS.md item M).
Transposition table, int-only entries	<code>dict</code> keyed by position, entry <code>(depth, score, flag, move_code)</code> with every field an <code>int</code> (the move is packed as from to $\ll$ 6 promotion $\ll$ 12 and rebuilt at the probe). Mate scores are stored ply-independent. Cap 250 000 entries; emptied at move start if above 60 % of the cap, and as a backstop when the cap is hit mid-search. Persists across moves within a game.	Worst generation-2 garbage-collection pause in a 60 s search: 143 ms (270k <code>chess.Move</code> entries) $\rightarrow$ 13.7 ms; RSS at the cap 143 MB $\rightarrow$ 120 MB; nps on that position 50.3k $\rightarrow$ 62.7k (source: RESULTS.md, “gen-2 gc pause” and “RSS at the TT cap”). A 143 ms stall is invisible to the clock check and would have been about three times longer on the platform’s core.

technique (continued)	what it does and why	measured effect (local, dev box)
Clock check every <code>NODE_CHECK_INTERVAL</code> = 128 nodes	Read <code>perf_counter()</code> and abort past the hard deadline or past an optional node limit.	With 1024 the search overshot the hard deadline by 25 ms mean, 65 ms worst locally (9 ms worst in the integration game); a clock read costs about 60 ns, and nps is unchanged at 128 (source: DECISIONS.md item A; RESULTS.md, “hard-deadline overshoot”).
Move ordering	TT move first; captures and promotions by MVV-LVA (victim value $\times 10$ minus attacker value; a promotion counts the promoted piece as the victim); two killer moves per ply (quiet moves that caused a cutoff at the same ply); remaining quiet moves by the history heuristic (bonus depth^2 per cutoff, indexed by colour, from- and to-square). Captures are never stored as killers.	Structural; no separate measurement.
Repetition inside the tree	A node whose position already occurred in the game history or on the current search path scores 0. The side that is ahead therefore avoids repeating and the side behind seeks it, with no root rule (source: DECISIONS.md, “Two-fold repetition”).	Pinned by <code>test_side_ahead_avoids_repetition</code> and <code>test_side_behind_seeks_repetition</code> .
Graph-history mitigation	A draw found through the path is a fact about the line, not the position, so a node whose value depended on one is not stored with its score; only its move is kept as an ordering hint at depth -1 , never over an entry earned without the repetition. Draws from the game history are permanent within the game and are stored normally.	Cost 0.15 % of nps (A/B, three 3 s searches each). Evidence: after a depth-7 search of 8/8/8/4k3/8/8/8/R3K3 w the table held a depth-5 lower bound of 0 for a position whose fresh value is -628 (source: DECISIONS.md item E).
Root tie-break, <code>DRAW_TIEBREAK_MARGIN</code> = 300	When the best root score is exactly 0, at least two root moves tie at it and the root static evaluation is at least 300 in our favour, the tied move whose child evaluates best is chosen (stalemates excluded): a rule draw inside the horizon must not stall a mop-up.	With the urgency clamps (Section ??) the 600-ply case 8/8/8/3K4/8/8/8/q6k b - - 0 291 converts (mate in 17 plies); it was drawn at the cap before (source: RESULTS.md, “Known unconverted wins”).
Queen over under-promotion on exact ties	If the first root move is an under-promotion, the queen promotion of the same pawn is searched before it, so the queen wins an exact tie (fail-soft scores of later moves are only bounds, so comparing them at the root was rejected).	The reviewer had seen <code>b8=R</code> chosen over <code>b8=Q</code> (source: DECISIONS.md item H).
Urgency clamps	Not a search technique but a budget one: when the 600-ply cap or, in a pawnless mop-up, the fifty-move draw is near, <code>moves_to_go</code> is clamped to the moves left before it so the search gets the time a mate needs (Section ??).	Two won KQ-vs-K games had been drawn by rule with the normal budget (source: DECISIONS.md item C).

6 Evaluation

Terms. The static evaluation (`mikhail_letal/evaluation.py`) has three terms: material, piece-square tables and a mop-up term. It is *tapered*: every piece has a middlegame table and an endgame table, and the position’s *phase* blends them. The phase counts the non-pawn pieces on the board with weights knight 1, bishop 1, rook 2, queen 4, so it is 24 with every piece on the board and 0 with only kings and pawns; the score is

$$\text{score} = \frac{\text{mg} \cdot \text{phase} + \text{eg} \cdot (24 - \text{phase})}{24},$$

truncated toward zero so that mirrored positions score exactly opposite. Material values are folded into the tables at load time, so the hot loop does one table lookup per piece per phase. Insufficient material scores 0. The result is negated for Black to move, so the search sees every node the same way. Piece bitboards are read directly rather than through `pieces_mask` and `scan_forward` (same semantics, 1.7× faster) (source: `INTEGRATION_NOTES.md`, “Builder deviations”).

Piece values. Pawn 100, knight 320, bishop 330, rook 500, queen 900, in both phases. These are the universal textbook values, recorded as such and untuned (source: `weights/PROVENANCE.json`).

The parametric prior. Each of the twelve tables (six pieces × two phases) is computed by `tools/gen_pst.py` from the square’s geometry. Two helper quantities: *centrality* is $3 - \max(|\text{file} - 3.5|, |\text{rank} - 3.5|)$, which is 2.5 on the four centre squares and −0.5 on the rim; *advancement* is the rank seen from the piece’s own side, 0 to 7. Then: pawns gain `pawn_adv_mg` per step forward and `pawn_centre_mg` on d/e files on ranks 3–5 in the middlegame, and `pawn_adv_eg`· $\text{advancement}^2/7$ in the endgame (a passed-pawn shape); knights gain `knight_centre` × centrality and lose `knight_rim` on the outer ring; bishops gain `bishop_centre` × centrality and `bishop_long_diagonal` on the two long diagonals; rooks gain `rook_seventh` on the seventh rank and `rook_centre_file` on d/e files in the middlegame and are flat in the endgame; queens gain `queen_centre` × centrality × 0.5 and lose `queen_early` for leaving the back two ranks in the middlegame; the king gains `king_shelter` on b1/c1/g1 and loses `king_centre_penalty` per rank of advancement and on the d/e files in the middlegame, and gains `king_centre_eg` × centrality in the endgame. The script prints every table as an 8×8 grid for a human to read, records the parameters and the tables’ SHA-256 in the JSON’s `_provenance` object, and a test checks that the shipped file matches the generator bit for bit (`test_pst_json_matches_the_generator`).

parameter	value	reason recorded in <code>tools/gen_pst.py</code>
<code>pawn_adv_mg</code>	4	each step forward gains space, but pawn storms weaken the king, so small
<code>pawn_centre_mg</code>	12	d/e pawns on ranks 3–5 hold the centre; comparable to a tempo
<code>pawn_adv_eg</code>	14	quadratic in steps forward ($14 \times 25/7 = 50$ on the 7th), a passed-pawn shape
<code>knight_centre</code>	12	knights gain most from the centre ($2.5 \times 12 = 30$) and lose most on the rim
<code>knight_rim</code>	14	a knight on the edge attacks half as many squares: extra penalty
<code>bishop_centre</code>	6	bishops act at range, so centrality matters half as much as for knights
<code>bishop_long_diagonal</code>	6	the two long diagonals see the whole board
<code>rook_seventh</code>	20	a rook on the 7th cuts off the king and eats pawns
<code>rook_centre_file</code>	8	d/e files are open most often in the middlegame
<code>queen_centre</code>	8	halved by the formula: the queen is strong everywhere
<code>queen_early</code>	12	a queen past the second rank invites harassment before the minors develop
<code>king_shelter</code>	25	castled squares b1/c1/g1 keep pawns in front of the king
<code>king_centre_penalty</code>	10	per rank of advancement and for the d/e files, so e8 gives -80 at most
<code>king_centre_eg</code>	10	the king becomes a fighting piece; $2.5 \times 10 = 25$ in the centre
<code>mopup_edge</code>	10	per Manhattan step of the bare king from the centre (max 6, so 60)
<code>mopup_close</code>	12	per step our king is closer than 14; exceeds one ring of <code>king_centre_eg</code>

Table 3: All sixteen parameters of the prior (`PARAMETERS` in `tools/gen_pst.py`, copied into `weights/pst.json` under `_provenance.parameters`). Magnitudes were chosen so the largest table entry stays under a pawn: piece-square effects refine material, they do not rival it. Resulting table ranges (min, max): middlegame P 0..24, N $-20..30$, B $-3..21$, R 0..28, Q $-14..2$, K $-80..25$; endgame P 0..50, N $-20..30$, B $-3..21$, R 0..0, Q $-2..10$, K $-5..25$ (source: `weights/PROVENANCE.json`).

Mop-up. When one side has a bare king, the other has no pawns and at least a rook’s worth of material (or two minor pieces), the stronger side gains $\text{mopup_edge} \cdot d_{\text{centre}}(\text{weak king}) + \text{mopup_close} \cdot (14 - d(\text{king, king}))$, with d the Manhattan distance: drive the bare king to the edge and bring ours close. The idea is textbook geometry; the two weights (10 and 12) are ours and were chosen by hand so that “close” beats one ring of the endgame king table (source: `DECISIONS.md`, 2026-09-06, “Mop-up term”).

Cost. `evaluate()` takes 3.4–3.8 μs in the middlegame and 1.6 μs in the endgame (10 000-call loops) (source: `RESULTS.md`, row “`evaluate()` cost”).

How the weights were obtained, and before/after. They were not tuned: Stage 0 ships the prior as generated, so there is no tuning script, data set or run to cite and no before/after arena result. Texel-style tuning of the piece values and tables on positions from our own arena games and public human games, labelled by game result or by a local reference engine’s static evaluation (permitted by the rules), is a Stage 2 experiment; it will be accepted only on a positive interval against the untuned set, and its script, data hash and run id will be added to Appendix ??.

7 Time management

The budget formula. Every constant lives in one frozen dataclass, `TimeParams` (file `mikhail_letal/timing.py`), so that a calibration changes one object and nothing else. With t

the clock in milliseconds before the move and m the number of moves we have played this game:

```

moves_to_go = clamp(40 -  $\lfloor m/2 \rfloor$ , 12, 40),
moves_to_go = min(moves_to_go,  $\lceil \text{plies\_to\_cap}/2 \rceil$ ,  $\lceil \text{fifty\_move\_room}/2 \rceil$ ),
soft =  $(t - \text{overhead})/\text{moves\_to\_go} + 0.8 \times 500$ ,
hard = min( $3 \times \text{soft}$ ,  $0.25 t$ ),
floor = max(1500,  $0.05 t$ ),
hard = min(hard,  $t - \text{overhead} - \text{floor}$ ),    soft = min(soft, hard),

```

both clamped to be non-negative. `plies_to_cap` = 600 – ply is always passed; `fifty_move_room` = 100 – halfmove clock is passed only in a pawnless mop-up, where nothing but the mate itself resets the halfmove clock (the urgency clamps of review item C). Worked examples from the unit tests: at $t = 120\,000$, $m = 0$: `moves_to_go` 40, `soft` = $119\,850/40 + 400 = 3396.25$ ms, `hard` = $10\,188.75$ ms; at $t = 60\,000$, $m = 20$: `moves_to_go` 30, `soft` = 2395 ms, `hard` = 7185 ms (source: `tests/test_timing.py`).

How the budget is used. `get_move` starts its own timer on entry, so board construction and history bookkeeping count against the budget as they do on the platform. The searcher is handed two absolute deadlines: a soft deadline at $0.45 \times \text{soft}$ after which no new iteration starts (the next iteration costs several times the previous one), and a hard deadline at which the search aborts mid-iteration; the clock is read every 128 nodes (Section ??). A node limit is supported by the search and tested (`test_node_limit_stops_search_close_to_the_limit`) but is not set in Stage 0; the brief’s node cap derived from a measured node rate is for the jitted search, where a misbehaving clock read is the risk it backs up. Below `panic_ms` = 1650 ms the engine is skipped and the fallback plays (Section ??); a position with a single legal move is played without searching.

constant	value	origin
<code>increment_ms</code>	500	agent contract
<code>overhead_ms</code>	150	brief’s initial guess; to be replaced by max over a game of (platform-measured move time – self-measured) + 50
<code>moves_to_go_max</code> / <code>min</code>	40 / 12	brief §6.2
<code>increment_fraction</code>	0.8	brief §6.2
<code>hard_multiplier</code> / <code>hard_fraction</code>	3.0 / 0.25	brief §6.2
<code>floor_ms</code> / <code>floor_fraction</code>	1500 / 0.05	brief §6.2
<code>panic_ms</code>	1650	<code>overhead_ms</code> + <code>floor_ms</code> : below it the formula has no time to plan with (the brief suggested about 1500; in the band 1500–1650 the formula returned a zero budget and a depth-0 move)
<code>next_iteration_fraction</code>	0.45	brief; to be replaced by the measured iteration-cost ratio

Table 4: Time-management constants of build v0.1.0, all uncalibrated (source: `docs/CALIBRATION.md`).

What the local games show. At the real clock the smoke games from the extracted zip ran 1.5 to 7.7 s per move against a soft target of about 3.3 s and a hard limit of about 10 s, with an average spend of 3.1–3.5 s per move including the increment (source: `RESULTS.md`, row “depth at 120 s clock”; `INTEGRATION_NOTES.md`, “Time profile”). The reason moves overrun the soft target is the iteration-cost ratio $T(d)/T(d-1)$: median 4.3–5.2 and up to about 10 on

middlegame searches, so an iteration begun at $0.45 \times \text{soft}$ usually runs to $2\text{--}2.5 \times \text{soft}$ and is cut by the hard limit; below about 20 s on the clock the hard limit ($0.25t$) binds every move (source: CALIBRATION.md, “Observations”). This is sustainable at 40 moves to go but is the first thing to check in the validation log. Worst hard-deadline overshoot inside `get_move`: 9 ms (`t 2661` against `h 2652`), with the 1024-node check interval then in force; the local harness’s own overhead (referee-measured minus self-measured) was 0–1 ms per move (source: RESULTS.md, rows “hard-deadline overshoot” and “local harness overhead”).

Calibration procedure. When a validation log arrives the operator pastes it and the following is done (source: CALIBRATION.md, “Procedure”): (1) extract the init time, every per-move time, the clock left after each move and any stderr; (2) pair each platform-measured move time with the self-measured `t` token of our log line; the difference is the platform overhead for that move, and `overhead_ms = max(difference) + 50`; (3) compute the platform node rate from our `nps` tokens and divide by the local node rate on the same build and positions to get the speed factor, and rescale any node cap; (4) record the platform init time against the local one; (5) commit the new constants with the log excerpt. The speed factor and the calibration table are **[PENDING: first platform validation log: table of local vs platform move times, init time, overhead and speed factor (CALIBRATION.md, “Log of calibrations”)]**.

8 Endgame handling

Mop-up. Section ??: a geometric term so that KX-vs-K converts without tablebases.

Tablebases. Not adopted in this build. Syzygy 3–4-man probing through `chess.syzygy` is a Stage 2 experiment; it ships only if the files fit inside the size cap with margin and an arena run from endgame-heavy positions shows a positive interval, otherwise the mop-up term stays (source: DECISIONS.md, 2026-09-06, “Mop-up term instead of tablebases”).

Draw handling. Inside the tree the terminal checks run in this order: `ply` ≥ 600 is a draw; a position already in the game history or on the search path is a draw; `halfmove clock` ≥ 100 is checkmate if the side to move is mated, otherwise a draw (the referee checks mate first, and so do we: `test_mate_in_one_beats_the_fifty_move_rule`). The game history starts at the first FEN received, as the referee’s does, and is kept by `GameState` (Section ??). Because any earlier occurrence scores a draw, the engine steers away from a repetition before the third occurrence when ahead, and toward one when behind. Near a rule deadline the urgency clamps (Section ??) and the root tie-break (Section ??) make sure a won position is not shuffled into a draw.

The 600-ply cap. `GAME_PLY_CAP = 600`; the `ply` is taken from the FEN’s move number, so the count matches the referee’s, which counts the opening position (source: tests: `test_ply_cap_scores_draw`).

position	outcome at 5 s/move	note
Lucena 1K1k4/1P6/8/8/8/8/r7/2R5 w - - 0 1 8/8/8/8/3k4/8/8/4KQ2 w - - 82 60 (KQ vs K, 18 plies of fifty-move room)	not converted: draw by fifty moves, mate two moves short	rook-endgame technique beyond a depth-6 horizon; Stage 2 the tablebase (queried by hand) gives mate in 13 plies, so it is winnable; the engine drives the king to the edge but not at tablebase pace, and depth 6 at 1 250 ms per move never sees the mate in time
8/8/8/3K4/8/8/8/q6k b - - 0 291 (KQ vs K, 19 plies before the 600-ply cap)	converted, mate in 9 moves (17 plies)	drawn at the cap before the 2026-09-07 fixes

Table 5: Endgame probes with the reviewer’s driver at 5 000 ms per move, where the hard limit ($0.25 \times$ clock) caps every move at 1 250 ms; the opponent is a second copy of the engine (source: RESULTS.md, “Known unconverted wins”). At the real 120 s clock the urgency clamps can add time; at 5 s they cannot.

9 Safety and failure handling

The wrapper. `agent.py` is deliberately small. `get_move` builds the board from the FEN, updates the game history, lists the legal moves, decides a budget, calls the engine, and checks the answer. The whole body is inside one `try`: any exception is logged in one line and the fallback plays. The engine’s move is accepted only if it is in the legal-move list; then a *final guard* (`_validated`) parses the UCI string back and looks it up in the legal moves of a fresh `chess.Board(fen)`, exactly as the referee will, replacing it with the fallback if that fails. Only after this check is the move recorded in the history and printed. Even the logging is wrapped, and lines are capped at 120 bytes because the platform keeps only the first and last 4 KB of output.

The fallback. `fallback_move` looks one ply ahead: a mate in one if there is one, otherwise the move that leaves us the most material, preferring big captures and queen promotions, with the UCI string as the last tie-break so the same position always gives the same move. It returns in a few milliseconds and is tested on its own (mate in one, best capture, promotion, en passant, determinism, legality in random positions, speed) (source: tests/test_fallback.py).

The panic band. Below `panic_ms = 1650` ms the engine is not called at all. The threshold equals `overhead_ms + floor_ms`: in the band 1500–1650 ms the formula returned a zero budget and the search aborted at its first clock check with a depth-0 move, so the band was routed to the fallback instead (source: INTEGRATION_NOTES.md, “For the reviewers” item 2; commit 974264d).

Desync reset. The platform sends nothing but a FEN and a clock. `GameState.observe` expects each received position to be one legal move away from the position after our last move, and reconstructs the opponent’s move by trying every legal move and comparing repetition keys. If no move matches (a desync: the platform and our history disagree), the history is reset to the received position, a line is logged, and the engine’s transposition table is cleared, because stored draws may rest on the history just discarded (source: `agent.py`; DECISIONS.md item E). One known gap is benign by design: when we play Black we never see the game’s true start

position, so its count stays one below the referee’s; since any earlier occurrence already scores a draw, nothing changes (`test_black_never_sees_the_start_position`).

Start-up. A depth-2 search from the initial position runs at import so every code path has executed before the clock starts; the table is cleared after it. `OMP_NUM_THREADS` and `NUMBA_NUM_THREADS` are set to 1 before any other import. Log lines are printed with `flush=True`: the runner points our stdout at a pipe, which Python block-buffers, and kills the process when the game ends; without the flush the first local game printed no agent output at all (source: `INTEGRATION_NOTES.md`, “What changed in integration”).

Fuzz counts. 300 games against `baselines/random` at 3 s + 0.05 s and 200 against `baselines/greedy` at 10 s + 0.1 s, over the collected openings, twelve games in parallel: 500 games, 500 wins by checkmate, zero terminations by crash, illegal move, flag or init (source: `RESULTS.md`, rows `fuzz-random-v0.1` and `fuzz-greedy-v0.1`). The random mover is the opponent that plays the strange moves (under-promotions, en passant, stalemate tries) that exercise the guard paths.

10 Testing methodology and results

Machines and load. Development machine: 16 cores, 7.8 GB RAM, WSL2 on Windows, Python 3.12.14. Platform: one core of an AMD EPYC 9V74 at 2.60 GHz, 2 GB (source: `RESULTS.md`, “Machine”). Every arena row records the one-minute load average at the start and the end of the run, because games sharing a machine stop measuring time management; the deciding games (real clock, previous version, strongest baseline, ≥ 40 games with the slowest move and minimum clock inspected) must be run alone.

Statistics. `tools/arena_openings.py` draws openings from `data/openings.txt` (219 distinct FENs under 34 names, collected from 400 public game pages of 80 of the 335 teams, one request per second, on 6 September) (source: `RESULTS.md`, row “curated openings collected”), alternates colours, and prints the same score, 95 % interval and Elo formulas as the starter harness, plus draw rate and termination counts. Rule of thumb for sample size (score near 50 %, about 25 % draws): the Elo standard error is about $300/\sqrt{n}$, so about 300 games resolve +50 Elo and about 1500 resolve +20; no improvement whose interval includes zero is ever claimed (source: brief, §7.3).

opponent	tc	games	+W =D -L	score	95 %	terminations	workers	load start / end
random	3 + 0.05 s	300	+300 =0 -0	100.0 %	±0.0 %	checkmate 300	12	0.10 0.27 0.22 / 7.45 2.31 0.93
greedy	10 + 0.1 s	200	+200 =0 -0	100.0 %	±0.0 %	checkmate 200	12	7.45 2.31 0.93 / 10.55 5.77 2.43
minimax	120 + 0.5 s	40	+39 =1 -0	98.8 %	±2.4 %	checkmate 39, threefold repetition 1	4	1.48 3.05 2.04 / 2.66 3.91 3.34

Table 6: Arena runs of build v0.1.0: rows **fuzz-random-v0.1**, **fuzz-greedy-v0.1** and **v0.1-vs-minimax-real-clock** of RESULTS.md, in that order; opponents are the starter’s **baselines/random**, **greedy** and **minimax**. Openings: **data/openings.txt**. The tool printed no Elo for these runs. Loads of 7–10 during the fuzz runs are expected with twelve workers and mean those games do not measure time management; the minimax run used four workers (load 1.5–3.9 on 16 cores), so its per-move times are indicative but it is not the solo pass of Stage 3. Its one draw is by threefold repetition; the PGNs are kept under **data/pgn/**. **baselines/greedy** searches one ply on material; **baselines/minimax** two plies on material and mobility with no time management (source: README.md).

measurement	value	how
node rate, middlegame	50–61k nps	agent log lines, harness.play vs greedy at 20 s + 0.2 s
depth at 20 s clock	4–6 (seldepth 13–21)	same game
depth at 120 s clock	5–6 (seldepth 18–32); 1.5–7.7 s per move vs soft $\approx$ 3.3 s, hard $\approx$ 10 s	harness.package smoke games from the extracted zip
hard-deadline overshoot inside get_move	9 ms worst seen	clock read every 1024 nodes at the time
local harness overhead	0–1 ms per move	referee clocks minus the t token
import time (incl. depth-2 warm-up)	0.033 s	three runs
peak RSS after a 10 s search	42.4 MB (16.8 MB before); 53 640 TT entries	resource.getrusage
evaluate() cost	3.4–3.8 μ s middlegame, 1.6 μ s endgame	10 000-call loops
zip	23 333 bytes, 66 825 unzipped	harness.package ; both smoke games clean
test suite	143 passed (44 property tests ran)	pytest -q , 33 s
review nps (3 s search, start position)	51.7k $\rightarrow$ 67.2k (+30 %), same move, depth 6/17	median of 3
gen-2 gc pause, 60 s search	143 ms max $\rightarrow$ 13.7 ms max	gc.callbacks timing
RSS at the TT cap	120 MB (250k entries) after; 143 MB (270k) before	same 60 s search

Table 7: Other measurements of build v0.1.0, development machine (source: RESULTS.md, “Other measurements”).

Static checks and smoke games. **ruff format -check**: 32 files already formatted; **ruff check .:** all checks passed; **mypy -strict**: no issues in 28 source files; **pytest -q**: 143 passed in 32.8 s; **harness.arena** vs random, 2 games at 5 s: +2 =0 -0 by checkmate; **harness.play** vs greedy at 20 s + 0.2 s: won by checkmate in 16 of our moves, twice; **harness.package**: zip built and both platform-style smoke games (120 s + 0.5 s, 20 plies) clean (source: INTEGRATION_NOTES.md, “Checks run”).

Perft. Not applicable to Stage 0 (Section 4); the Stage 1 table will go here.

Per-move log of the minimax run. The arena tool cannot see the agent’s log, so three further games were played alone on the machine with `harness.play` (two as White, one as Black, all won by checkmate): 71 engine moves, one of them 2 ms over its hard cap (the clock is read every 128 nodes), slowest move 8 429 ms against a cap of about 10 s, minimum clock before a move 42 958 ms, mean 2 526 ms per move against a mean soft budget of 2 925 ms, depth 1–7 (median 5), median 64 270 nps, import 31–34 ms (source: RESULTS.md, “v0.1 solo real-clock pass”). The one repetition draw of the 40-game run was a position where the engine was a pawn down (material 20 vs 21) and steered into the repetition, which is the intended behaviour when behind..

Final measurement pass. [PENDING: Stage 3 final pass alone on the machine at 120 s + 0.5 s: vs the previous version, vs the strongest baseline, and ≥ 40 games with the slowest move and the minimum clock reached].

11 Rating estimate

Everything in this section is a method statement; the numbers are [PENDING: Stage 3 rating estimate: yardstick matches, MLE fit, and the four-scale conversion]. The statement will be tagged as an estimate.

Why the ladder’s anchors cannot be mapped to CCRL directly. Five house engines hold fixed ladder ratings and also have published CCRL Blitz (40/4) ratings. The two sets do not agree even in order.

house engine	CCRL Blitz 40/4 (fetched 6 Sep 2026)	ladder anchor
Loki 3.0.0	2427 $\pm$ 13	1721
Zagreus 5.0	2168 $\pm$ 16	1686
Rustic Alpha 3.0.0	1791 $\pm$ 16	1630
Shallow Blue 2.0.0	1576 $\pm$ 19	1632
Zagreus 2.3.1	1399 $\pm$ 20	1527
spread	1 028	194

Table 8: The five fixed-rating house engines (source: brief, §8.1).

A 1 028-point CCRL spread compressed to 194 ladder points, not in CCRL order, with similar win–loss records, means the organisers are almost certainly running these engines throttled (fixed depth or nodes or similar) and set the anchor values by their own calibration. We therefore do not derive a ladder-to-CCRL mapping from the anchors; both sets of numbers are recorded here and that is all.

Yardstick matches (local only). The instrument is `tools/yardstick/agent.py`, a harness-compatible agent that drives a UCI engine through `chess.engine.SimpleEngine` on the real clock it is handed, with `Threads=1`, `Hash=16`, `Move Overhead=100`, `UCI_LimitStrength=True` and `UCI_Elo` at a level taken from an environment variable. The engine is a locally installed Stockfish, whose `UCI_Elo` is documented as calibrated at 120 s + 1 s and anchored to CCRL 40/4, close to the competition clock; the same wrapper without strength options runs the real Sunfish as a second yardstick. Levels 1400, 1600, 1800, 2000 and 2200 (extended upward if the

agent scores over 70 % at 2200), at least 60 games per level at 120 s + 0.5 s, colours alternated, openings from `data/openings.txt`, machine load noted. Fit: the maximum-likelihood rating R over all games with expected score $1/(1 + 10^{(L-R)/400})$ against level L and draws counted as one half; a bootstrap over games for the 95 % interval; then the yardstick’s own calibration uncertainty (± 100) added in quadrature. Reported as “CCRL-40/4-scale $\approx R$ (95 % interval ...)”. Alongside it: the harness’s Elo-vs-baselines numbers, the score against the real Sunfish, and, once the operator has ladder results, the ladder rating, which pins one point of the ladder-to-CCRL map at our own strength.

Conversion to human scales, two independent routes. Engine-pool Elo does not map cleanly to human ratings; a difference of X between two engines is not a difference of X between two humans, and the mapping is not linear, so two routes are used and the union of their ranges is quoted. *Route A (engine pool to FIDE)*: from the CCRL-scale R , the TalkChess rule of thumb $FIDE \approx 2800 - 0.7 \times (2800 - CCRL)$ as the upper end (it is stated to break down below about 1800 CCRL) and $FIDE \approx CCRL - 100$ as the lower end; low confidence. *Route B (human-anchored bot)*: the real Sunfish bot on lichess holds a rapid rating of 1808 against humans, running on PyPy with pondering; on CPython without pondering about 100 lower is assumed, so about 1700 lichess rapid. Our measured Elo difference against Sunfish is compressed by 0.7–1.0 for the engine-to-human transfer and added to that anchor, then converted lichess rapid $\rightarrow$ FIDE and $\rightarrow$ chess.com with the ChessGoals July-2026 comparison table (about 20 000 profiles; its FIDE column has no reliable data below about 1550 FIDE). The chess.com Rapid and Blitz ranges come from the same table by interpolation between these points: FIDE 1740 $\leftrightarrow$ blitz 1500 / rapid 1655; 1865 $\leftrightarrow$ 1800 / 1870; 1965 $\leftrightarrow$ 2000 / 1995; 2075 $\leftrightarrow$ 2200 / 2110; 2190 $\leftrightarrow$ 2400 / 2215; 2300 $\leftrightarrow$ 2600 / 2300. Ranges, never a point.

Final statement (to be filled in). CCRL-scale $\approx$ **[PENDING: R (interval)]**; FIDE-equivalent $\approx$ **[PENDING: $a-b$]**; chess.com Rapid-equivalent $\approx$ **[PENDING: $c-d$]**; chess.com Blitz-equivalent $\approx$ **[PENDING: $e-f$]**. These are estimates built on (i) a rating-limited reference engine, (ii) one human-rated bot and (iii) a survey-based table; the honest uncertainty is ± 200 or more on the human scales.

12 Known limitations and next stage

- **Lucena.** The rook-endgame technique of 1K1k4/1P6/8/8/8/8/r7/2R5 w is beyond a depth-6 horizon; not converted at 5 s per move (source: RESULTS.md, “Known unconverted wins”).
- **KQ vs K at halfmove clock 82.** 8/8/8/8/3k4/8/8/4KQ2 w - - 82 60 is winnable (mate in 13 plies with 18 of room) but is drawn by the fifty-move rule at 5 s per move: the engine makes steady progress but not at tablebase pace, and is two moves from mate when the rule lands. At the real clock the urgency clamps can buy time; at 5 s the hard limit caps every move at 1 250 ms (source: DECISIONS.md item D).
- **TT clear stall.** When the table hits its 250 000-entry cap mid-search it is emptied before the next store: crude but safe. The clear at move start above 60 % keeps this from landing mid-iteration in normal play, but it happened once in the 60 s stress search (source: RESULTS.md, “RSS at the TT cap”). An ageing replacement scheme is Stage 1 work.
- **Pure-Python speed.** 50–67k nps and depth 5–6 at the real clock is the ceiling of this architecture; the platform’s core is slower than the development machine by a factor that is **[PENDING: speed factor from the first validation log]**.
- **Uncalibrated time constants** until the first validation log; the soft target is overrun by design (Section ??) and the first log decides whether that is safe.

- **No tuning and no ablations.** The evaluation is the untuned prior and no search technique has an Elo ablation yet; both are on the plan below.
- **Private-API key.** `Board._transposition_key()` is private to python-chess; safe at the pinned 1.11.2, replaced by our own Zobrist keys in Stage 1.

Next stage. Stage 1 is the numba engine: our own board representation, move generation, make/unmake, evaluation and search compiled with numba and called from the same wrapper, with python-chess kept as the legality oracle and fallback. Its definition of done: perft agreement with python-chess to depth 4 on the standard positions and depth 3 on every collected opening and ≥ 2000 random positions, identical legal-move sets on $\geq 100\,000$ random positions; Stage 0’s fuzz and property tests passing unchanged; import time from the zip ≤ 40 s; ≥ 300 games against v0.1 at $10\text{ s} + 0.1\text{ s}$ with the 95 % interval entirely above zero, then ≥ 100 games at the real clock, alone on the machine, zero flags and interval above zero (source: brief, §5). Inside it: principal-variation search, null-move pruning, late-move reductions, aspiration windows, mate-distance pruning and a fixed-size numpy transposition table, each measured. Stage 2 then adds evaluation terms, Texel tuning, the tablebase experiment and time-management refinements, one arena run each.

A Provenance of every constant

Copied from two files: `docs/PROVENANCE.md`, which lists the search and timing constants that live in code, and `weights/PROVENANCE.json`, which lists the tables and ships in the zip. The generator commit is 4109710 (full hash in the JSON); the tables’ SHA-256 is recorded in `weights/pst.json` under `_provenance.tables_sha256`.

Table 9: Provenance table. Rules: every generating script is deterministic and re-runnable; a hand-adjusted value says so in its note with the reason; a weight file’s header names its script and data hash.

parameter / file	value or shape	produced by	data	date	note
piece values (mg, eg)	P 100, N 320, B 330, R 500, Q 900, K 0	<code>tools/gen_pst.py</code> @ 4109710	none (text-book)	2026-09-06	universal textbook values, recorded as such; to be tuned in Stage 2
<code>weights/pst.json</code> PSTs	6 pieces $\times$ 64 $\times$ {mg, eg}; ranges in Section ??	<code>tools/gen_pst.py</code> @ 4109710 (parametric geometric prior; parameters in the file’s <code>_provenance</code>)	none (formula)	2026-09-06	not copied from any engine; printed and eyeballed; one coincidental row vs CPW (Section ??)
mop-up weights	edge 10, close 12	<code>tools/gen_pst.py</code>	none	2026-09-06	chosen by hand from the geometric idea
phase weights	N 1, B 1, R 2, Q 4 (total 24)	<code>tools/gen_pst.py</code> $\rightarrow$ <code>pst.json</code> (<code>evaluation.py</code> only validates them)	none	2026-09-06	standard tapered-evaluation weighting
<code>MATE_SCORE</code> , <code>MATE_THRESHOLD</code>	100 000, 99 000	code	none	2026-09-06	arbitrary large constants

parameter / file (continued)	value or shape	produced by	data	date	note
TT entry cap	250 000 (default of <code>Engine</code>); <code>dict</code> , cleared when full and at move start above 60 % (<code>TT_CLEAR_FRACTION</code>)	code	none	2026-09-07	bounds memory: 120 MB RSS at the cap; int-only entries keep gc pauses ≤ 14 ms
<code>NODE_CHECK_INTERVAL</code>	128	code	none	2026-09-07	clock cadence; 1024 gave 25 ms mean / 65 ms max overshoot locally; a read costs about 60 ns
<code>MAX_PLY</code>	128	code	none	2026-09-06	recursion bound for search, quiescence and extensions
check extension	+1 ply when in check	code	none	2026-09-06	textbook; applied before the TT probe
<code>QS_EVASION_PLIES</code>	4	code	none	2026-09-07	keeps eight-queens-a-side positions under 100k nodes at depth 1
<code>DRAW_TIEBREAK_MARGIN</code>	300	code	none	2026-09-07	“clearly ahead” = at least a minor piece
history heuristic bonus	depth ²	code	none	2026-09-06	textbook
<code>GAME_PLY_CAP</code>	600	code	none	2026-09-06	the referee’s cap
time constants	Section ?? table	code	none	2026-09-06	brief §6.2 initial values; calibration [PENDING: first validation log]